

01 // AI / ENGINEERING

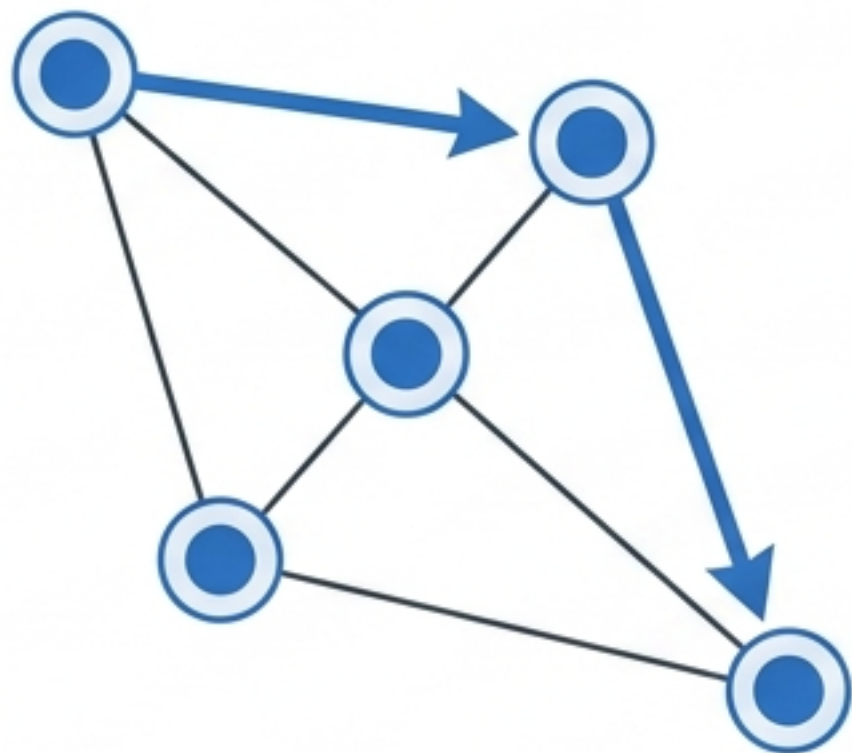
Harness-інженерія для AI-агентів

Як перетворити хаос великої кодової бази
на кероване середовище для LLM.

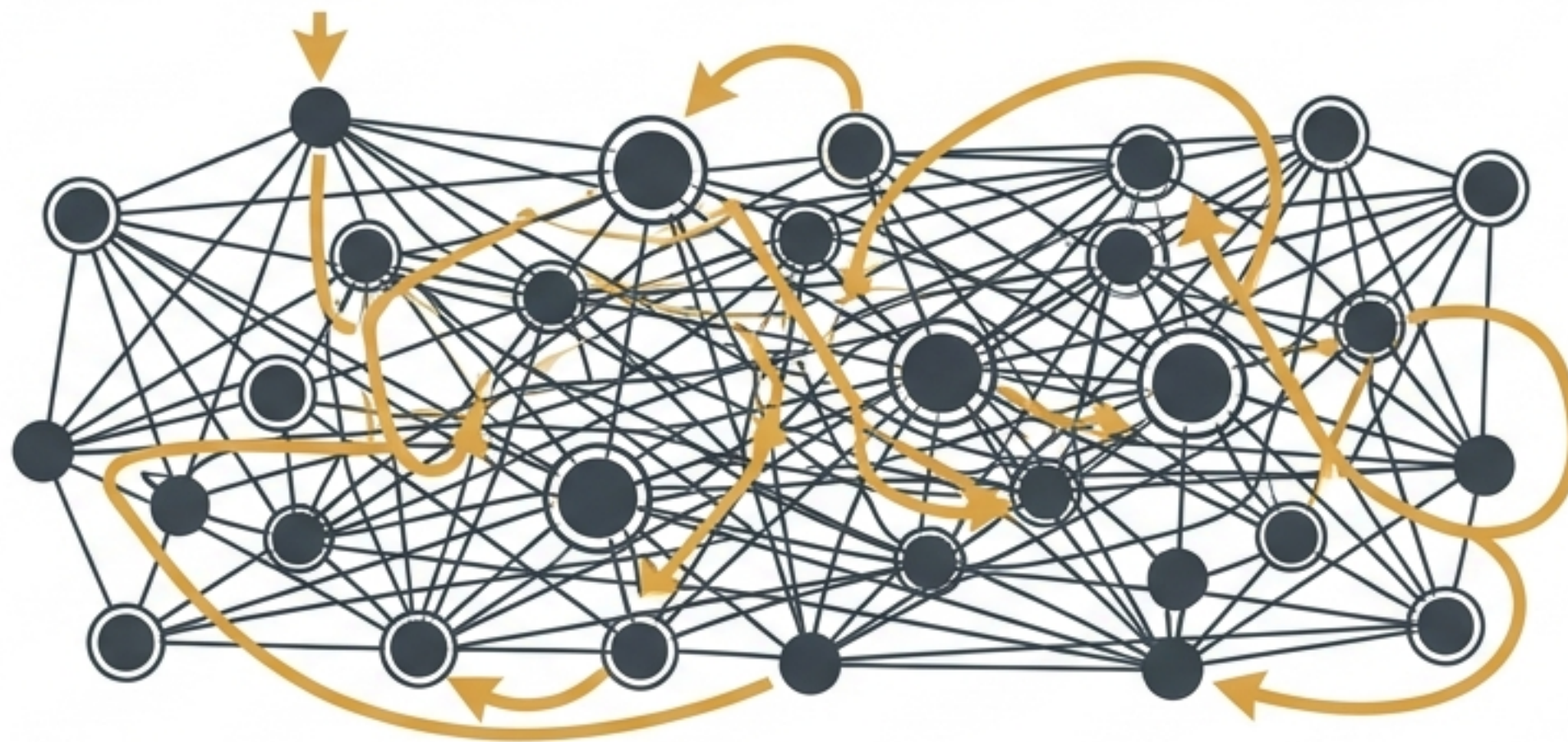
У великому репозиторії ШІ неминуче губиться

На маленькому проєкті все як на долоні. Але в масштабній системі агент починає блукати, перечитувати файли і витрачати обмежений ресурс — контекст.

Малий проєкт



Монолітний репозиторій



Контекст — це найдорожчий і найбільш обмежений ресурс агента. Без навігації він спалюється на пошук, а не на код.

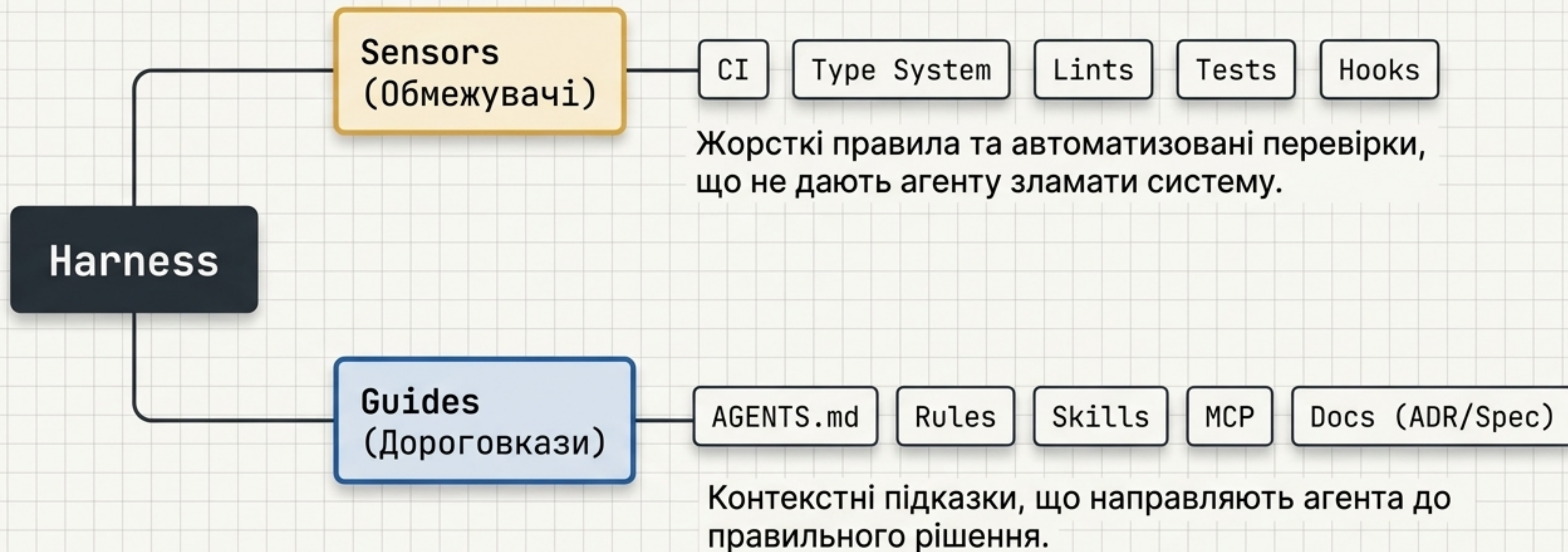


"Двигун"
Аналітичні здібності,
генерація коду.

"Шасі та кермо"
Середовище, інструкції,
обмеження.

Фундаментальний концепт, описаний **Martin Fowler** та **OpenAI Codex**. Великим репозиторієм потрібен фреймворк для орієнтації.

Анатомія Harness-інженерії

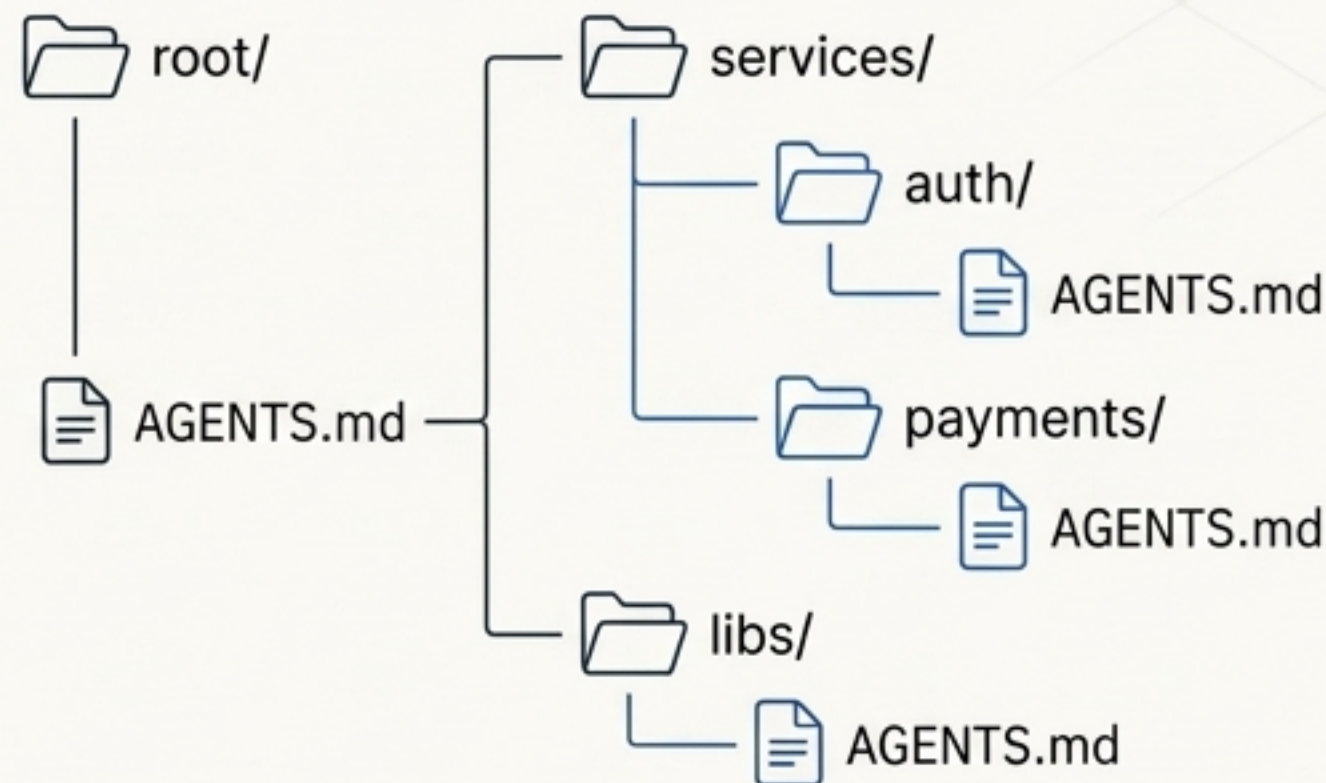


AGENTS.md — найдорожчий файл у репозиторії

Це Always-on контекст. Кожен запит агента містить ці інструкції. Ціна кожного рядка тут — максимальна.

AGENTS.md

```
1 # Загальні інструкції агенту
2
3 ## Роль
4 Ви – старший інженер. Дотримуйтесь принципів SOLID.
5
6 ## Стандарти коду
7 – Завжди використовуйте типізацію.
8 – Покривайте код тестами.
9 – Уникайте глобальних змінних.
10
11 ## Процес
12 1. Аналізуйте вимоги.
13 2. Проектуйте рішення.
14 3. Імплементуйте з тестами.
15
16 ## Контекст проекту
17 Проект використовує архітектуру мікросервісів. Основна мова –
18 Go. База даних – PostgreSQL. Комунікація через gRPC.
19
20 # Критичні правила
21 Ніколи не видаляйте дані без підтвердження. Завжди
22 перевіряйте вхідні дані. Зберігайте зворотну сумісність.
```



Best Practice

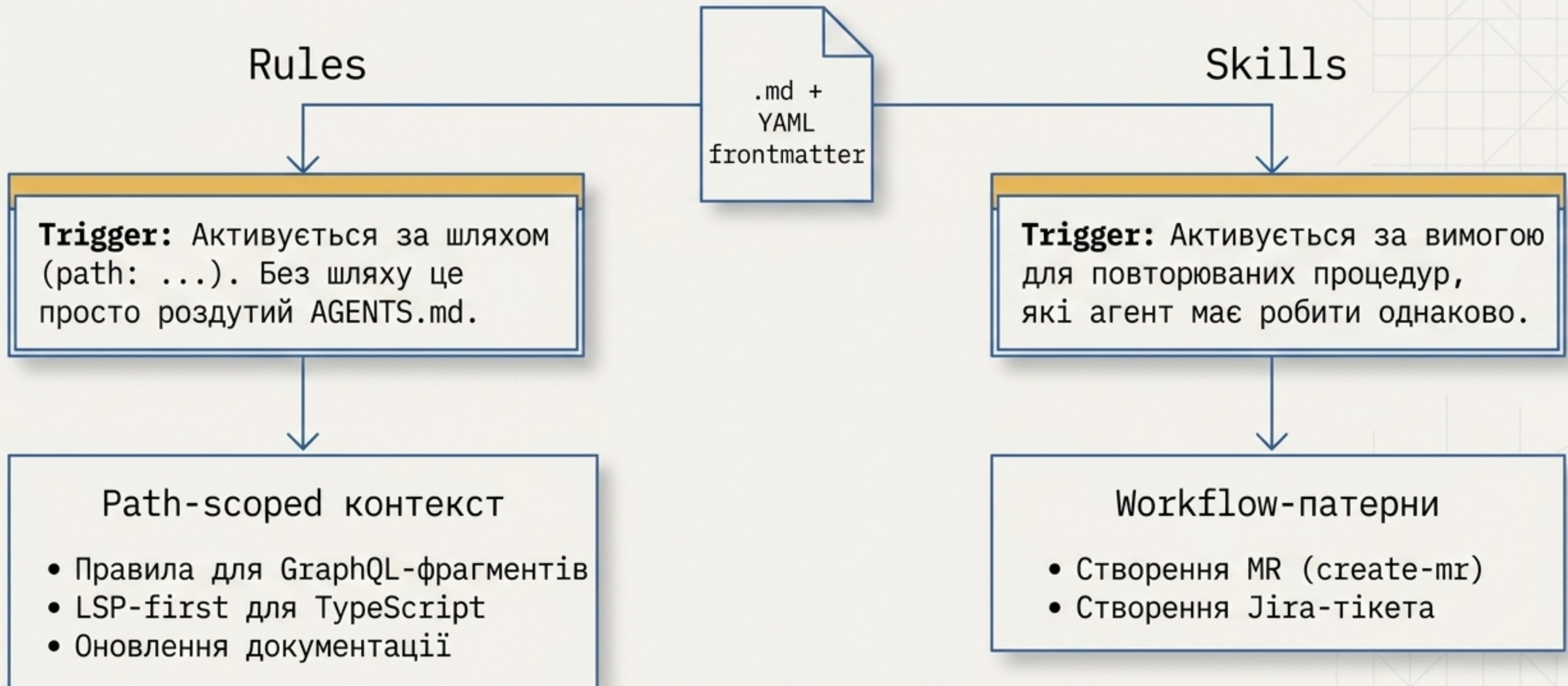
Візуалізація дерева тек. Замість одного гігантського файлу в корені — використання вкладених AGENTS.md для локального контексту лише там, де він потрібен.

Фундаментальний концепт, описаний Martin Fowler та OpenAI Codex.
Великим репозиторіям потрібен фреймворк для орієнтації.

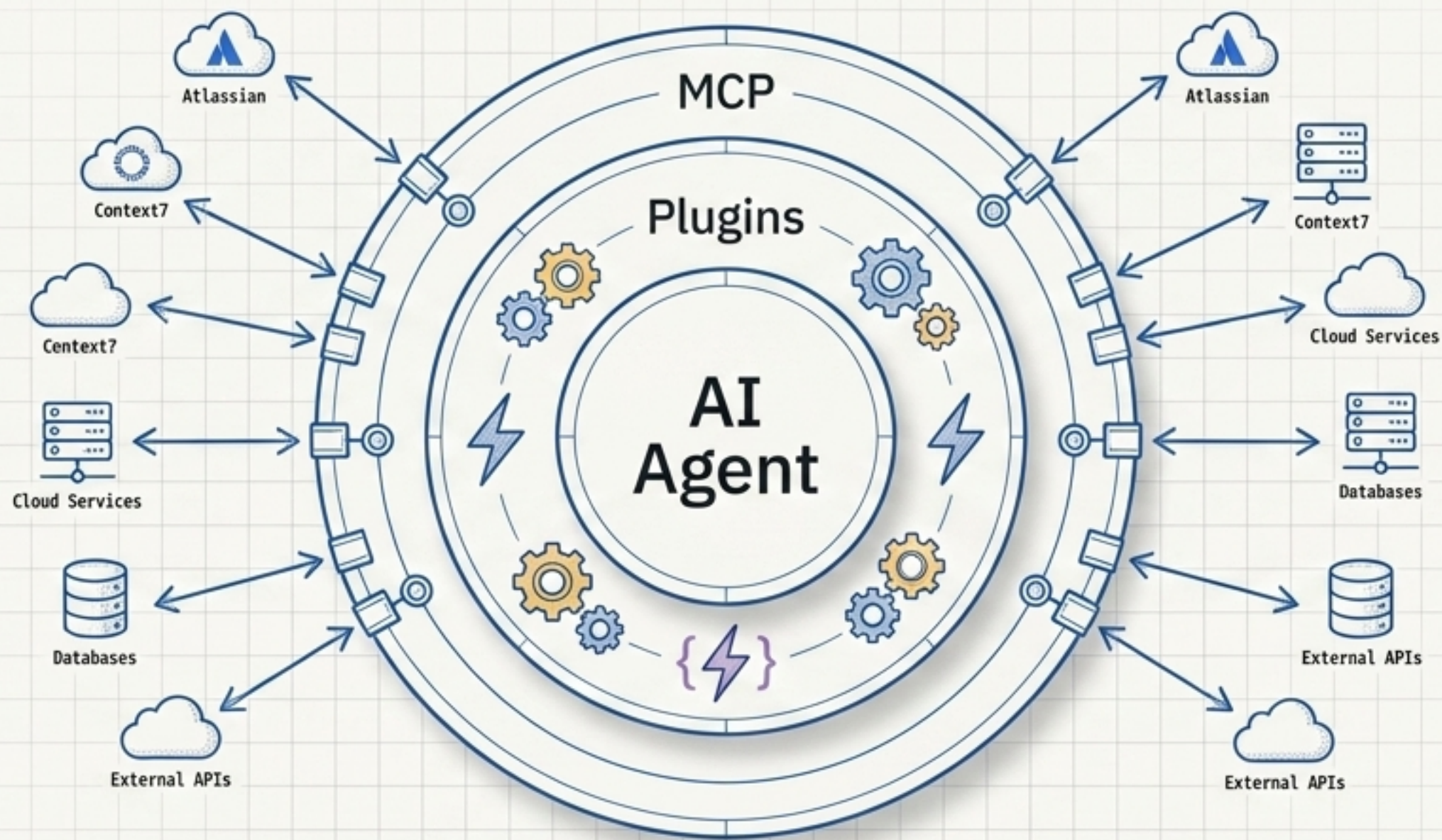
Матриця оптимізації глобального контексту

Працює (Доведено даними)	Антипатерни (Вбиває ефективність)
✓ Конкретний tooling: Вказівки на <code>uv</code>, <code>pytest</code>, <code>rdm</code> та скрипти. (Найкорисніша частина). ✓ Лише мінімум: Тільки критичне для роботи. ✓ Результат: Runtime -28.6%, Output-токени -16.6%.	✗ Огляд структури репо: Агент перечитує ті самі файли. (-20% success rate). ✗ Згенеровано ШІ (/init): Написано не людиною. (-3% success, +20% вартості). ✗ Зайві 'думати'-правила: Дрібний стиль, дублювання docs/. (+22% токенів на 'роздуми').

Динамічний контекст: Rules проти Skills



Розширення меж: MCP та плагіни



MCP (Model Context Protocol)

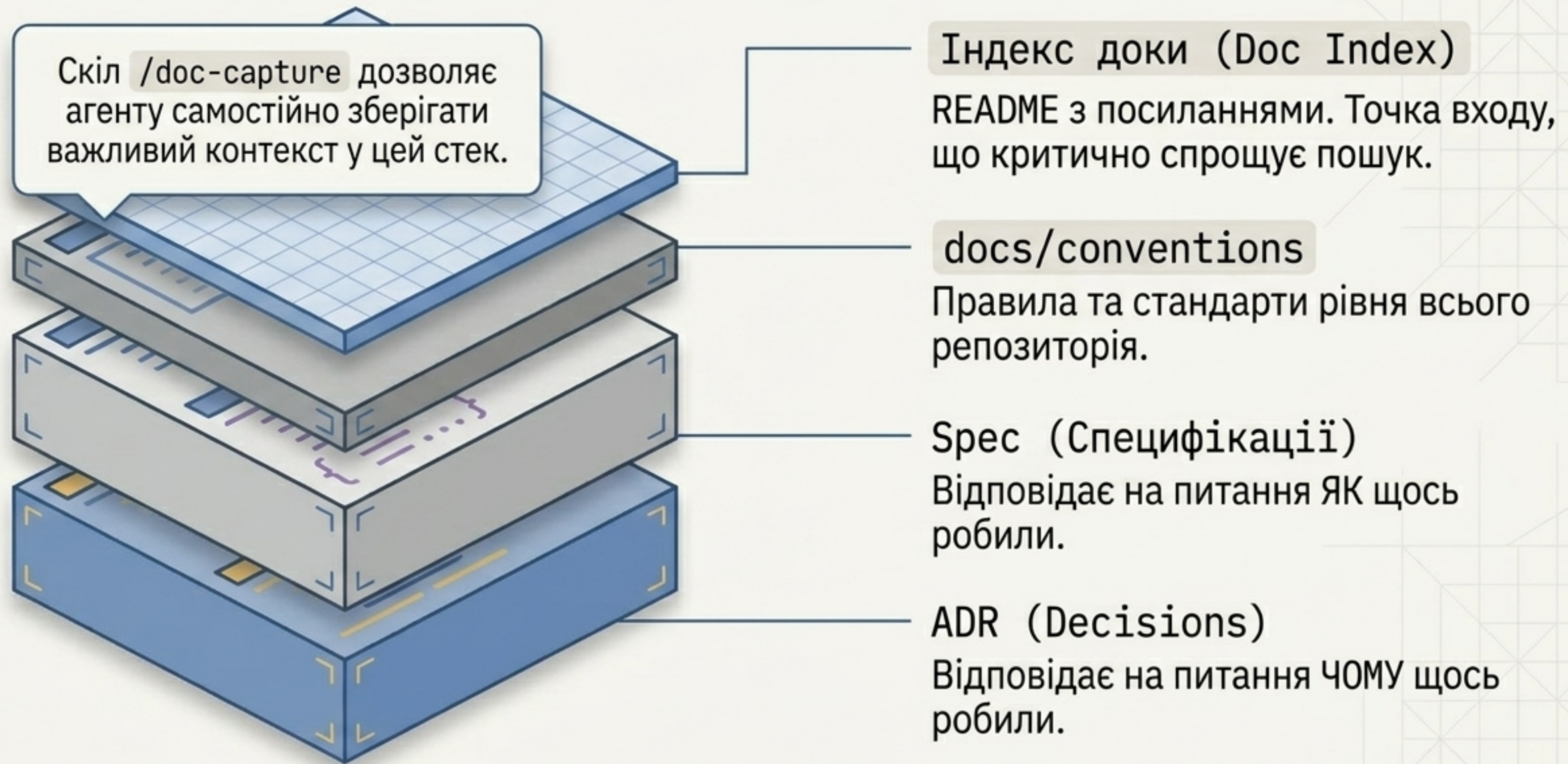
Протокол для підключення зовнішніх інструментів та джерел даних (наприклад, інтеграція з Atlassian, context7).

Plugins (Набори скілів)

Конвеєри розробки, упаковані як Superpowers.

[brainstorm] [TDD] [subagent] [worktree] [code review]

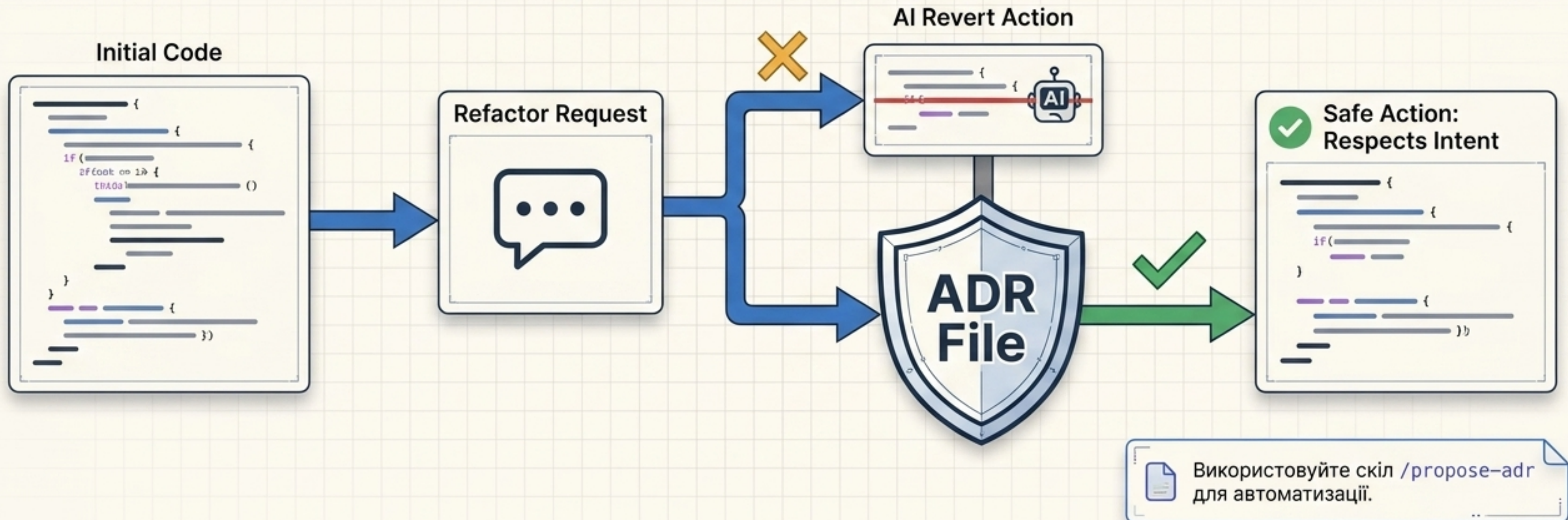
Документація як довготривала пам'ять агента



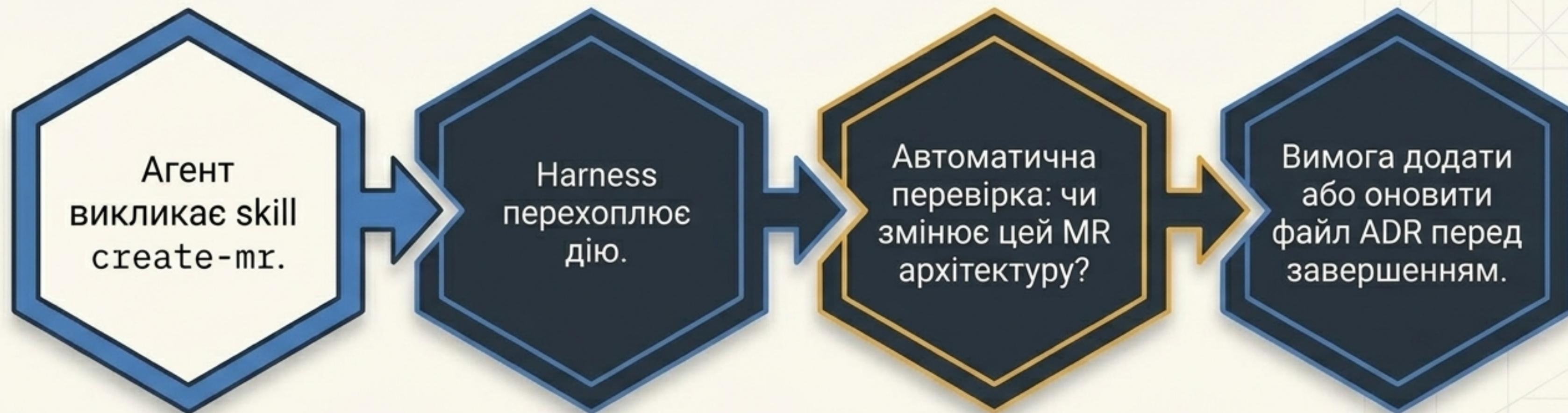
Фокус на ADR: Захист контексту 'Чому'

Одне рішення = один файл. Immutable (тільки superseded).

Код показує "що" зроблено. Без ADR агент не бачить "чому". Він вважатиме навмисний "костиль" помилкою і "виправить" його назад, ламаючи логіку. З ADR агент поважає межу.



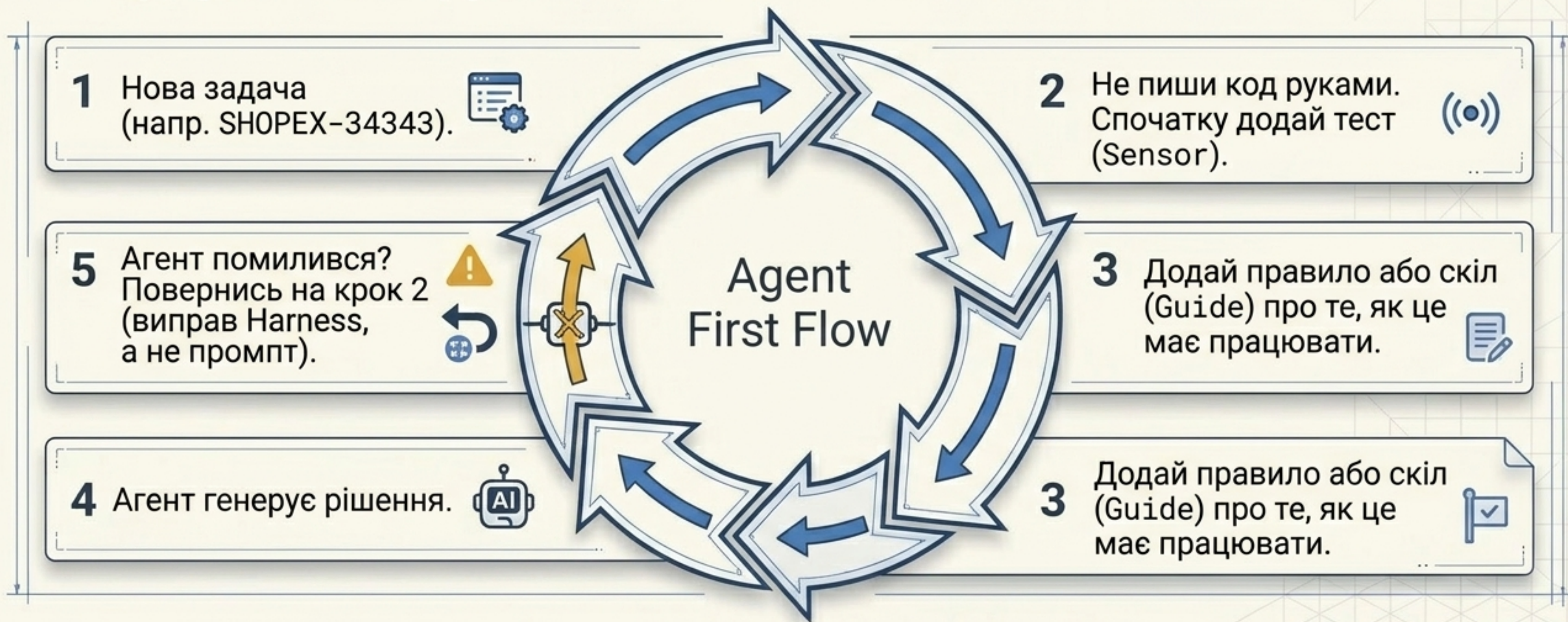
Кейс-стаді: Інтеграція в процес catalog-ui



Harness — це не лише інструкції, це **автоматизовані процеси**, що страхують агента.

Культура розробки: Agent-First Flow

**ШІ-агент стає точкою входу для БУДЬ-ЯКОГО workflow.
Ми будуємо інструменти для агента, а агент пише код.**



Інженерний Кодекс Harness

1. Harness будують інженери, а не модель. Згенеровані ШІ правила шкодять системі.
2. Пріоритет: спочатку будуємо Sensors (тести, CI, лінтери), і лише потім – Guides (інструкції та правила).
3. Агент повторює помилку? Це не поганий ШІ. Це сигнал докрутити Harness.



t.me/ainomadic